

Quantum Computing and Qiskit Practice



Qiskit?



Qiskit is an SDK for performing quantum computations that utilize these quantum mechanical principles using the language of quantum circuits.

Install <https://docs.quantum-computing.ibm.com/start/install>

1. Create a minimal environment with only Python installed in it.

```
python3 -m venv c:\path\to\virtual\environment
```

Windows

```
python3 -m venv /path/to/virtual/environment
```

Linux or macOS

2. Activate your new environment.

```
c:\path\to\virtual\environment\Scripts\Activate.ps1
```

Windows

```
source /path/to/virtual/environment/bin/activate
```

Linux or macOS

3. Install pip

```
C:> py -m ensurepip --upgrade
```

Windows

```
python -m ensurepip --upgrade
```

Linux or macOS

4. Install qiskit

```
pip install qiskit
```



IBM Quantum Lab

The screenshot displays the IBM Quantum Lab web interface. On the left is a sidebar with a file explorer. At the top of the sidebar is a 'New file +' button and a search bar labeled 'Filter files by name'. Below the search bar, the section 'Lab files /' contains a table of files. The table has two columns: 'Name' and 'Last Modified'. The files listed are: a folder (2 months ago), a folder (3 months ago), a folder (3 minutes ago), a folder (3 minutes ago), a folder (a month ago), a folder (3 months ago), and a file named 'Untitled.ipynb' (9 hours ago). The 'Untitled.ipynb' file is selected. The main area of the interface has a menu bar with 'File', 'Edit', 'View', 'Run', 'Kernel', 'Tabs', 'Settings', and 'Help'. Below the menu bar, there are two tabs: 'Untitled.ipynb' and 'Launcher'. The 'Launcher' tab is active, showing three options under the 'Notebook' section: 'Python 3 (ipykernel)', 'Getting started with Qiskit', and 'Python 3 (ipykernel)'. Below the 'Notebook' section is a 'Console' section with one option: 'Python 3 (ipykernel)'. At the bottom is an 'Other' section with four options: 'Text File', 'Markdown File', 'Python File', and 'Show Contextual Help'.

Name	Last Modified
[Folder]	2 months ago
[Folder]	3 months ago
[Folder]	3 minutes ago
[Folder]	3 minutes ago
[Folder]	a month ago
[Folder]	3 months ago
Untitled.ipynb	9 hours ago

API Token

Welcome, [REDACTED]



Graphically build circuits with
IBM Quantum Composer

Launch Composer



Develop quantum experiments in
IBM Quantum Lab

Launch Lab

Jump back in:

[REDACTED]

API token ⓘ

[View account details](#)

Check

```
import qiskit.tools.jupyter
%qiskit_version_table
```

Version Information		
Qiskit Software		Version
qiskit-terra		0.24.1
qiskit-aer		0.11.2
qiskit-ibmq-provider		0.20.0
qiskit		0.41.0
System information		
Python version		3.9.13
Python compiler		MSC v.1929 64 bit (AMD64)
Python build		tags/v3.9.13:6de2ca5, May 17 2022 16:36:42
OS		Windows
CPUs		6
Memory (Gb)		7.721988677978516
Tue Jul 04 10:28:29 2023 대한민국 표준시		



Quantum Computing and Qiskit Practice

Quantum Problem and Quantum Computer



“I'm not happy with all the analyses that go with just classical theory, because Nature isn't classical, dammit, and if you want to make a simulation of Nature, you'd better make it quantum mechanical, and by golly it's a wonderful problem!”



What Do Quantum Researchers do?

* Difficult Points

Make **Hamiltonian** and **Find Eigenstates/Eigenvalues,** especially **Ground State/Energy**

1. Make it Well-defined
2. Be Solvable in Life-time
3. Excited State is more harder



Caution

- **Quantum Computer**

A term referring to a hardware system capable of quantum calculations using qubits.

- **Quantum Computing**

A term referring to the quantum computation process or algorithms themselves that utilize quantum algorithms.



Quantum Computing and Qiskit Practice

Quantum Notation

Hat($\hat{\quad}$): Operator $\rightarrow$ Matrix

Bra($\langle \quad |$): State $\rightarrow$ Row Vector

Ket($|\quad\rangle$): State $\rightarrow$ Column Vector

Examples

$$\hat{A}|a\rangle = |b\rangle \longrightarrow \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

$$\langle a|b\rangle \longrightarrow (a_1 \quad a_2) \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = a_1 b_1 + a_2 b_2$$

$$|a\rangle\langle b| \longrightarrow \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} (b_1 \quad b_2) = \begin{pmatrix} a_1 b_1 & a_1 b_2 \\ a_2 b_1 & a_2 b_2 \end{pmatrix}$$

Time-independent Schrodinger Equation

$$\hat{H}|\psi\rangle = E|\psi\rangle$$



Quantum Computing and Qiskit Practice

Qubit and Gates

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$\begin{aligned} |a\rangle_1 |b\rangle_2 &= \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \otimes \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \\ &= c_{00}|00\rangle + c_{01}|01\rangle + c_{10}|10\rangle + c_{11}|11\rangle \\ &= \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix} \end{aligned}$$

Gates(Operators)

Single-qubit gates

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Two-qubit gates

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Examples

$$\begin{pmatrix} 1 \\ 1 \end{pmatrix} = |0\rangle + |1\rangle \rightarrow \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

$$H^n |0\rangle^{\otimes n}$$

$$\begin{pmatrix} \boxed{1} & \boxed{0} & 0 & 0 \\ \boxed{0} & \boxed{1} & 0 & 0 \\ 0 & 0 & \boxed{0} & \boxed{1} \\ 0 & 0 & \boxed{1} & \boxed{0} \end{pmatrix} \begin{pmatrix} c_{00} \\ c_{01} \\ c_{10} \\ c_{11} \end{pmatrix} = c_{00}|00\rangle + c_{01}|01\rangle + \overset{\curvearrowright}{c_{11}|10\rangle} + c_{10}|11\rangle$$



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
from qiskit import *  
  
import matplotlib.pyplot as plt  
%matplotlib inline  
  
qc = QuantumCircuit(1)  
qc.draw('mpl')
```

q —



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

QuantumRegister

```
qr = QuantumRegister(1)
qc = QuantumCircuit(qr)
qc.draw('mpl')
```

q_0 —

ClassicalRegister

```
qr = QuantumRegister(3)
cr = ClassicalRegister(3)
qc = QuantumCircuit(qr, cr)
qc.draw('mpl')
```

q_{2_0} —

q_{2_1} —

q_{2_2} —

c_0 ³ —



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
qc = QuantumCircuit(3, 3)  
qc.draw('mpl')
```

q_0 —

q_1 —

q_2 —

c ³ //



Quantum Computing and Qiskit Practice

Qiskit Practice

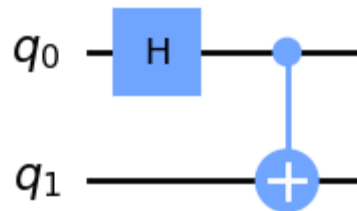
Let's Make a Bell State

$$|\text{Bell}\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

$$|0\rangle_0 |0\rangle_1 \xrightarrow{\text{Hadamard to 1st qubit}} \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl')
```



Quantum Computing and Qiskit Practice

Qiskit Practice

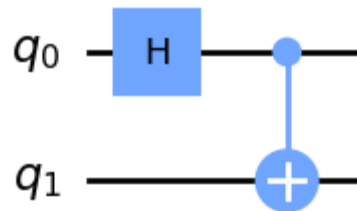
Let's Make a Bell State

$$|\text{Bell}\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

$$|0\rangle_0 |0\rangle_1 \xrightarrow{\text{Hadamard to 1st qubit}} \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \xrightarrow{\text{CNOT}} \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

```
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

qc.draw('mpl')
```



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
from qiskit.visualization.array import array_to_latex
```

Statevector

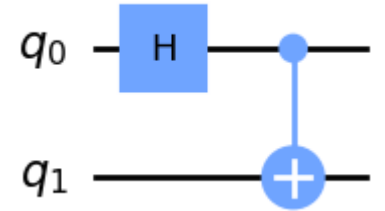
```
from qiskit.quantum_info import Statevector
```

```
array_to_latex(Statevector(qc))
```

StatevectorSimulator

```
from qiskit_aer import StatevectorSimulator
```

```
backend = StatevectorSimulator()  
array_to_latex(backend.run(qc).result().get_statevector())
```



$$\left[\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2} \right]$$

$$\left[\frac{\sqrt{2}}{2}, 0, 0, \frac{\sqrt{2}}{2} \right]$$



Quantum Computing and Qiskit Practice

Qiskit Practice

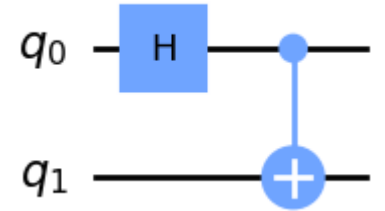
Let's Make a Bell State

AerSimulator

```
from qiskit_aer import AerSimulator

qc.save_statevector()
backend = AerSimulator()
backend.run(qc).result().data(0)['statevector']
```

```
Statevector([0.70710678 + 0.j, 0.+0.j, 0.+0.j, 0.70710678 + 0.j], dims = (2, 2))
```

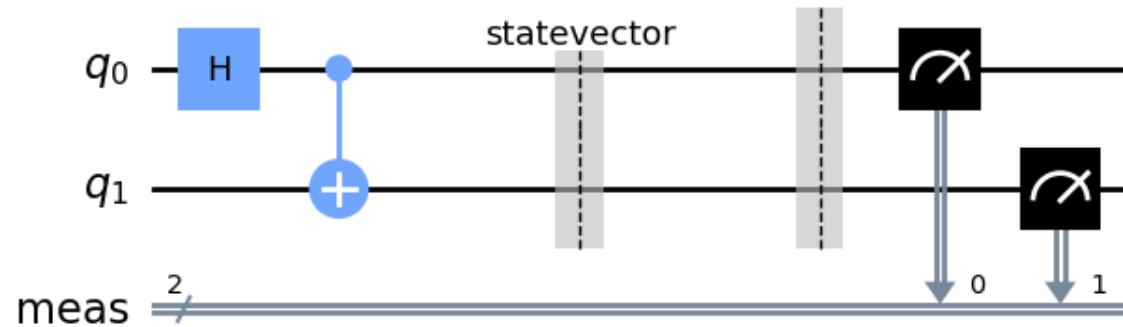


Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
qc.measure_all()  
qc.draw('mpl')
```



```
from qiskit.visualization import plot_histogram
```

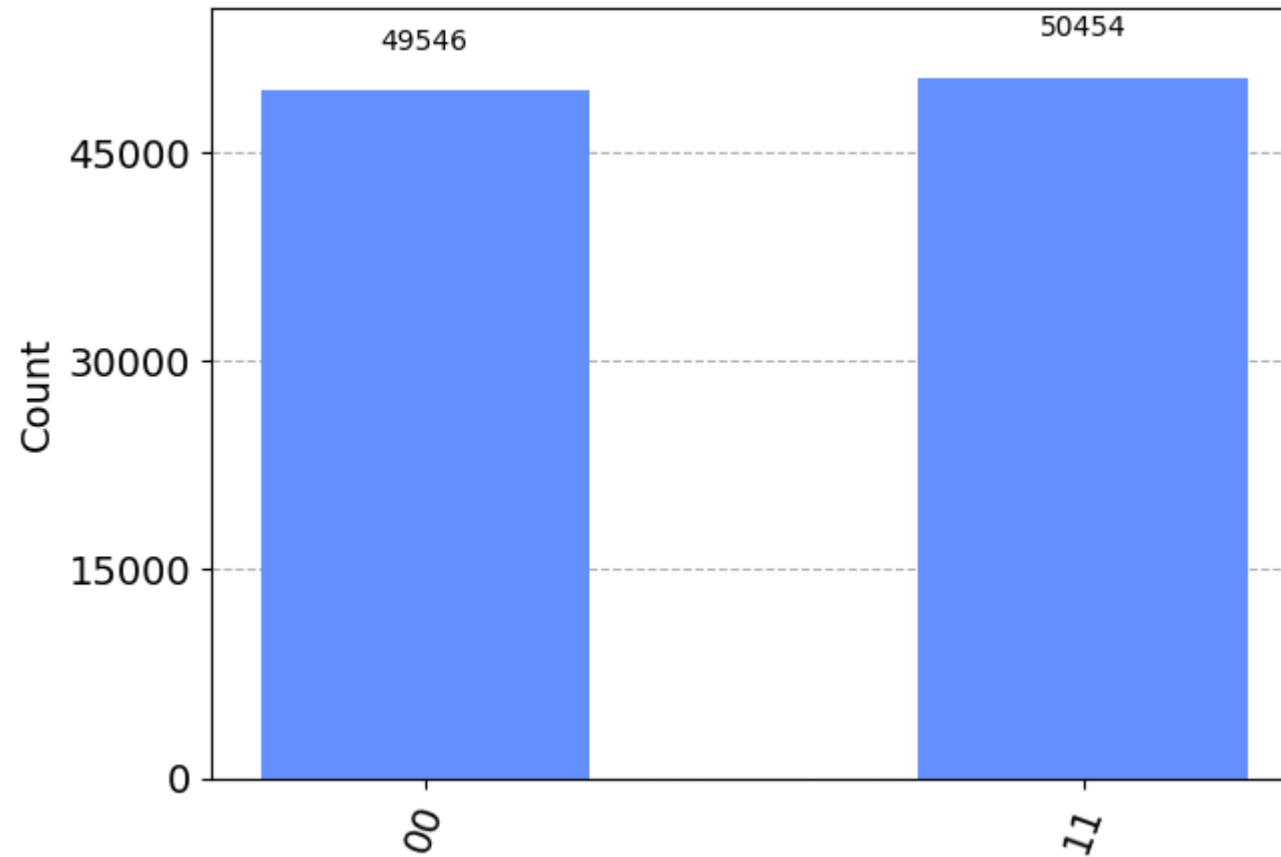
```
backend = AerSimulator()  
counts = backend.run(qc, shots=100000).result().get_counts()  
plot_histogram(counts)
```



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
from qiskit.providers.fake_provider import FakeVigo
device_backend = FakeVigo()

sim_noise = AerSimulator.from_backend(device_backend)

tcirc = transpile(qc, sim_noise)

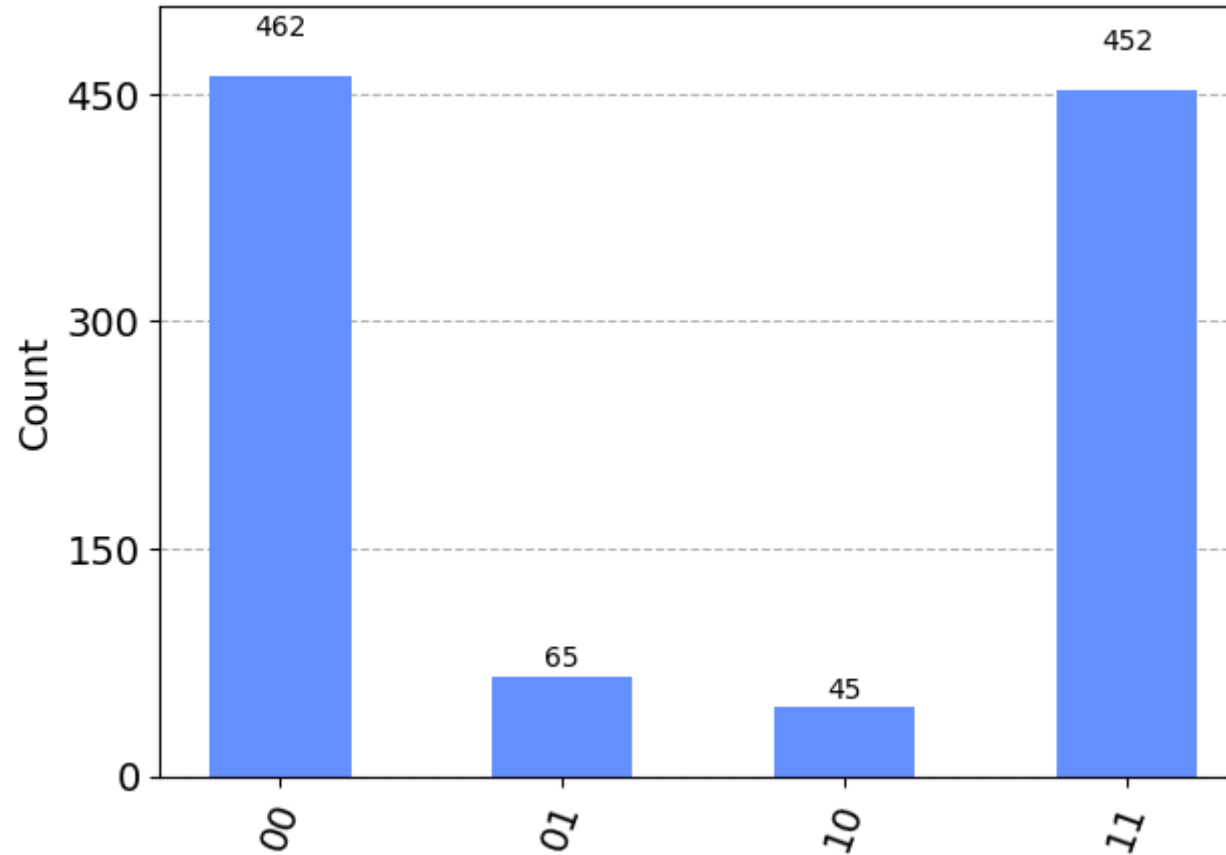
result_noise = sim_noise.run(tcirc).result()
counts_noise = result_noise.get_counts(0)
plot_histogram(counts_noise)
```



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State



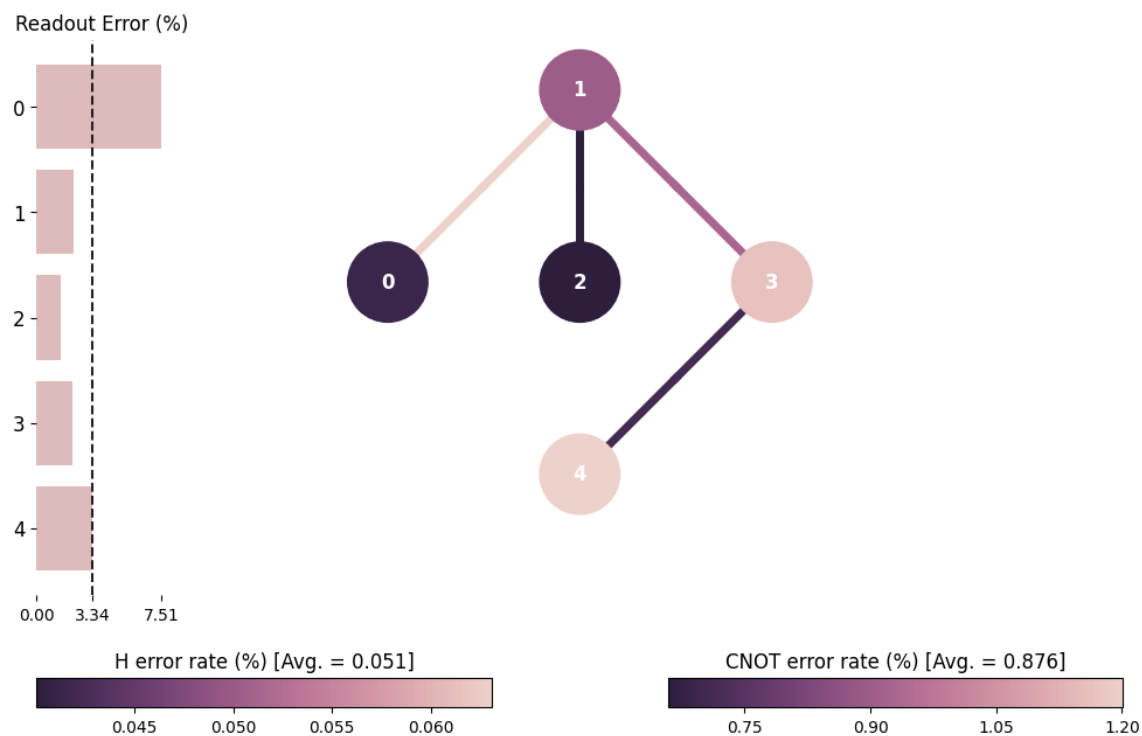
Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
from qiskit.visualization import plot_error_map  
plot_error_map(device_backend)
```

fake_vigo Error Map

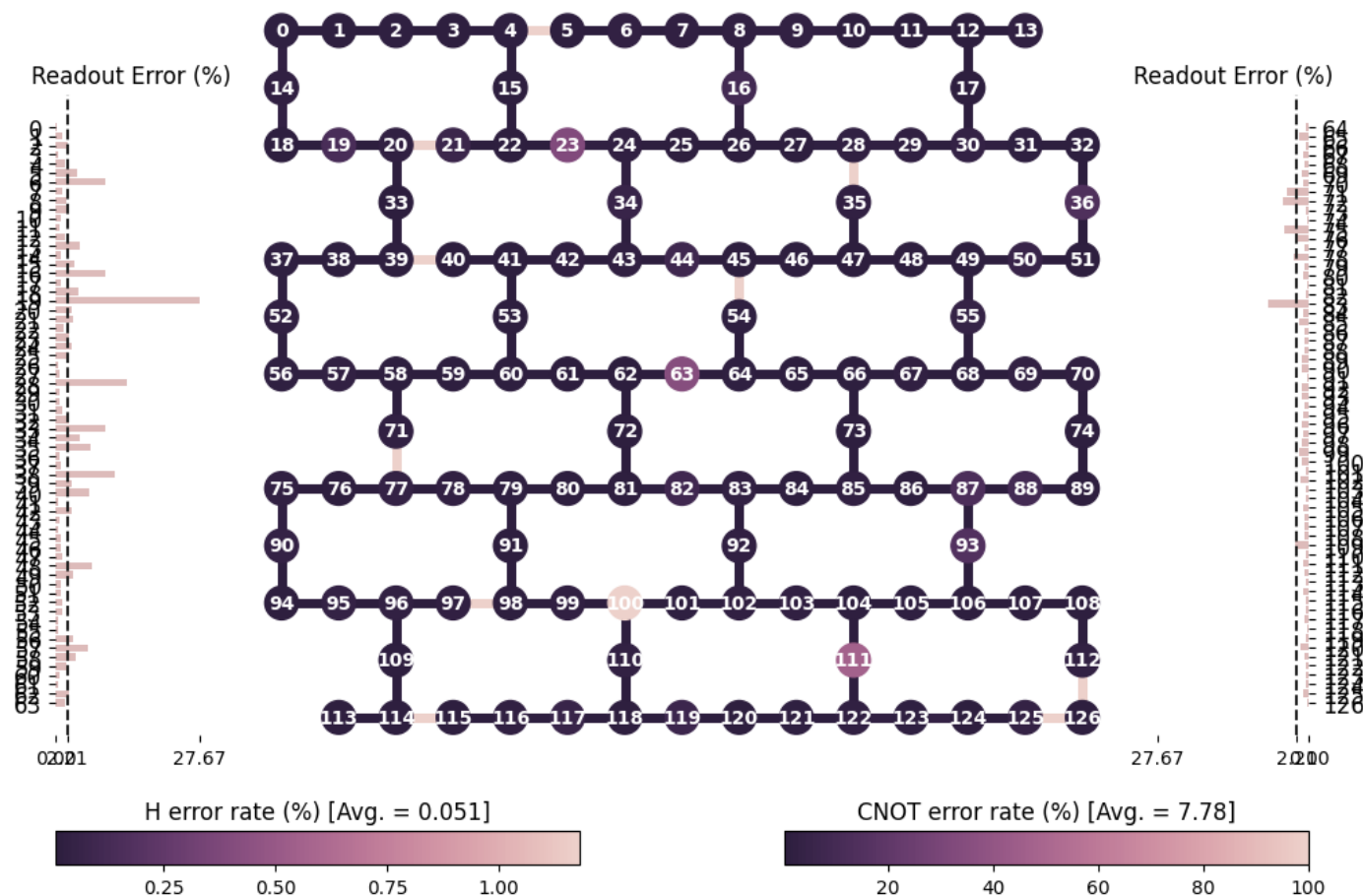


Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

ibm_sherbrooke Error Map



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
from qiskit_ibm_provider import IBMProvider
from qiskit_ibm_provider import least_busy

provider = IBMProvider()

backend = least_busy(provider.backends())
backend

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

qc_rb = transpile(qc, backend)
real_job = backend.run(qc_rb, shots = 10000)
real_job.job_id()
```



Quantum Computing and Qiskit Practice

Qiskit Practice

Let's Make a Bell State

```
job = provider.retrieve_job('#####')  
job.status()
```

```
result_real = job.result()  
counts_real = result_real.get_counts()  
plot_histogram(counts_real)
```

Jobs	
Search jobs	
	████████████████████ ID ██████████ ibm_cairo
	████████████████████ ID ██████████ ibm_cairo
	████████████████████ ID ██████████ ibm_cairo
	████████████████████ ID ██████████ ibmq_qasm_simulator
	████████████████████ ID ██████████ ibm_sherbrooke

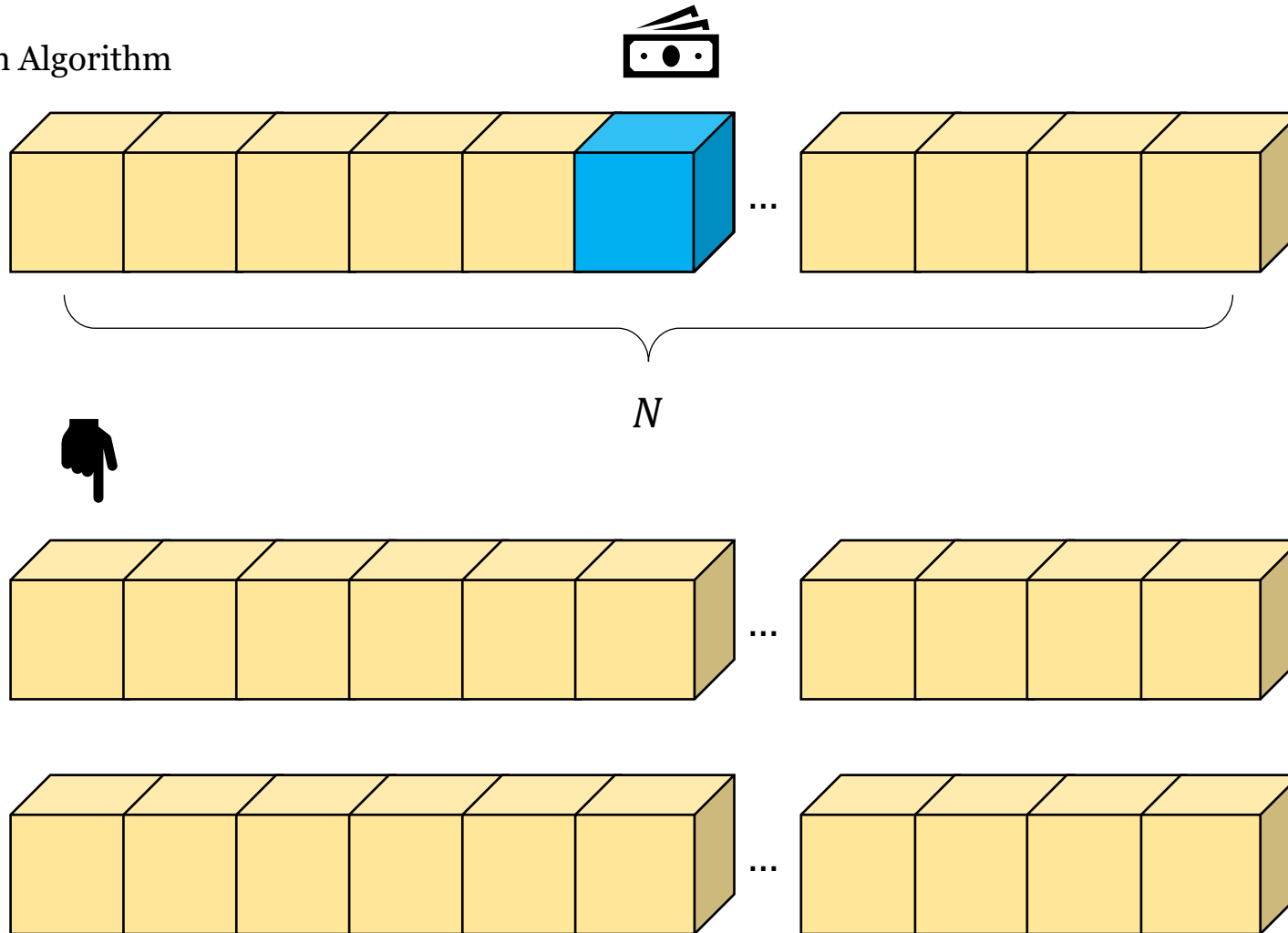


Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm

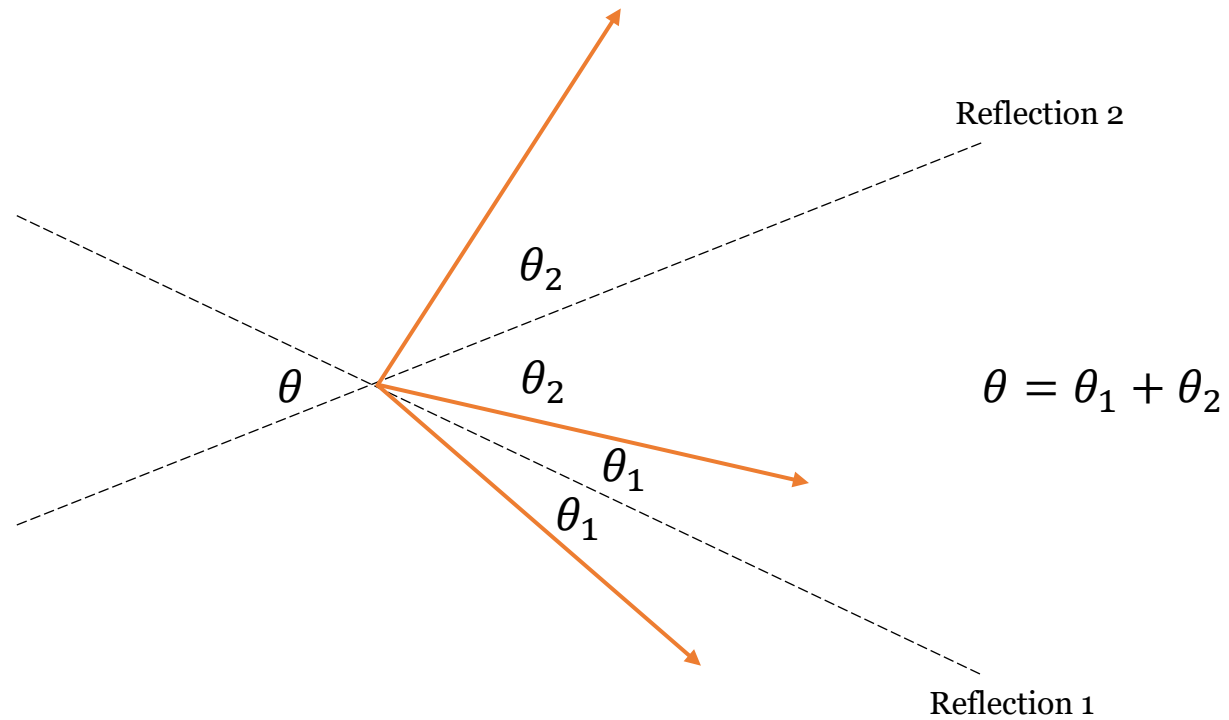
Search Algorithm



Quantum Computing and Qiskit Practice

Quantum Algorithms

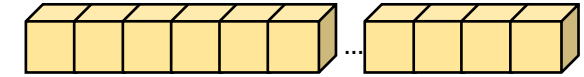
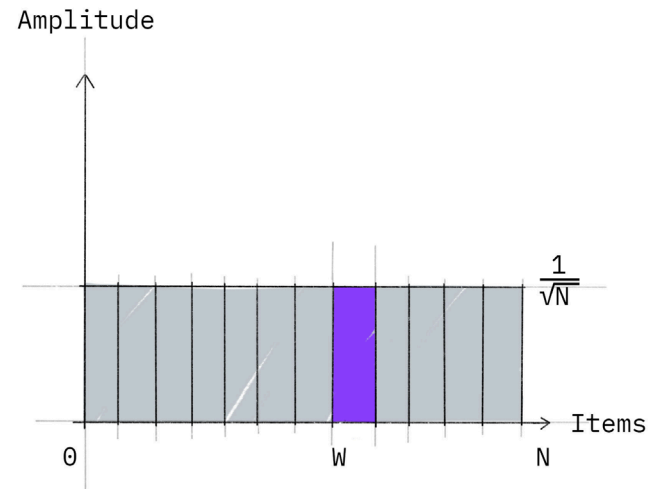
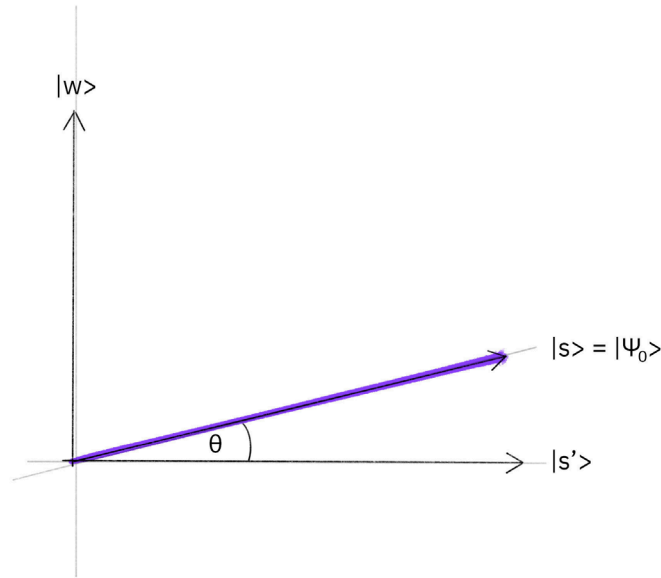
Grover's Algorithm



Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm



$$|00 \dots 0\rangle \sim |11 \dots 1\rangle$$

$$|s\rangle = \sin \theta |w\rangle + \cos \theta |s'\rangle$$

$$\theta = \arcsin \langle s | w \rangle = \arcsin \frac{1}{\sqrt{N}}$$

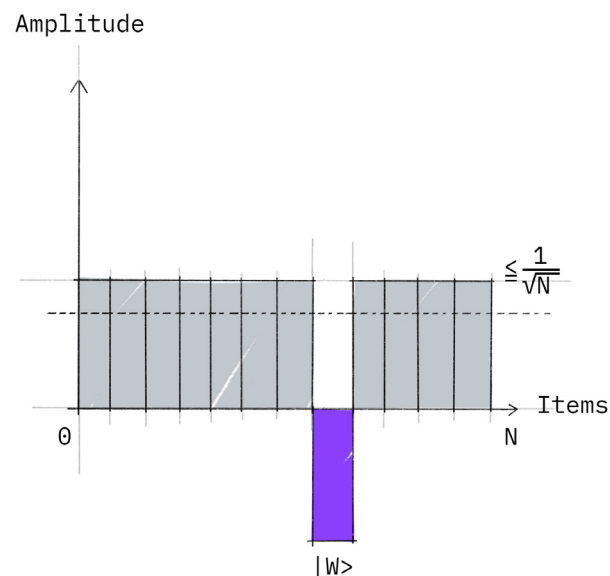
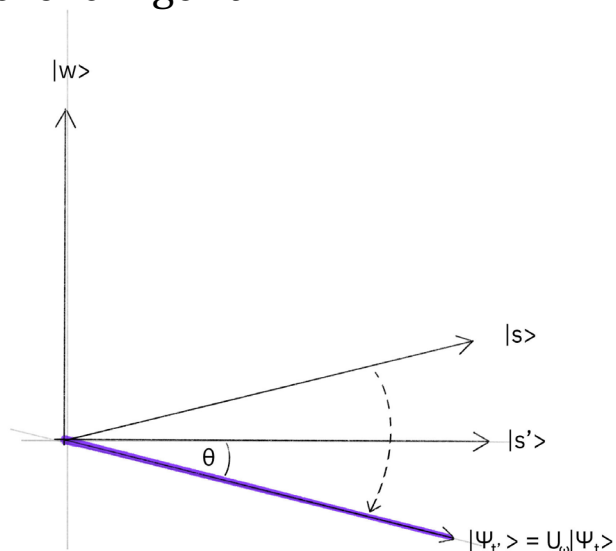
$$|s\rangle = \frac{1}{\sqrt{N}} \left(|w\rangle + \sum_{i \neq A} |i\rangle \right) = \frac{1}{\sqrt{N}} |w\rangle + \frac{\sqrt{N-1}}{\sqrt{N}} |s'\rangle = \sin \theta |w\rangle + \cos \theta |s'\rangle$$



Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm



$$U_f |s\rangle = (-1) \sin \theta |w\rangle + \cos \theta |s'\rangle$$

$$U_f = I - 2|w\rangle\langle w|$$

Oracle

$$U_f |w\rangle = |w\rangle - 2|w\rangle\langle w|w\rangle = -|w\rangle$$

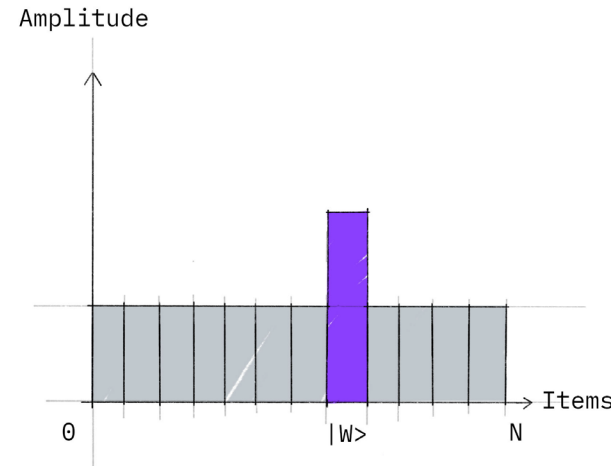
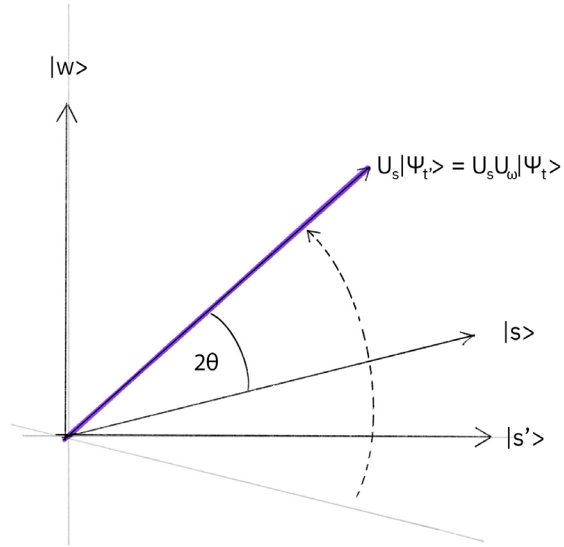
$$U_f |s\rangle = |s\rangle - \frac{2}{\sqrt{N}} |w\rangle$$



Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm



$$U_s = 2|s\rangle\langle s| - 1$$

Diffuser

$$U_s U_f |s\rangle = (2|s\rangle\langle s| - 1) \left(|s\rangle - \frac{2}{\sqrt{N}} |w\rangle \right) = |s\rangle - \frac{2}{\sqrt{N}} |s\rangle - \frac{2}{\sqrt{N}} |w\rangle = \frac{N-4}{N} |s\rangle - \frac{2}{\sqrt{N}} |w\rangle$$



Quantum Computing and Qiskit Practice

Quantum Algorithms

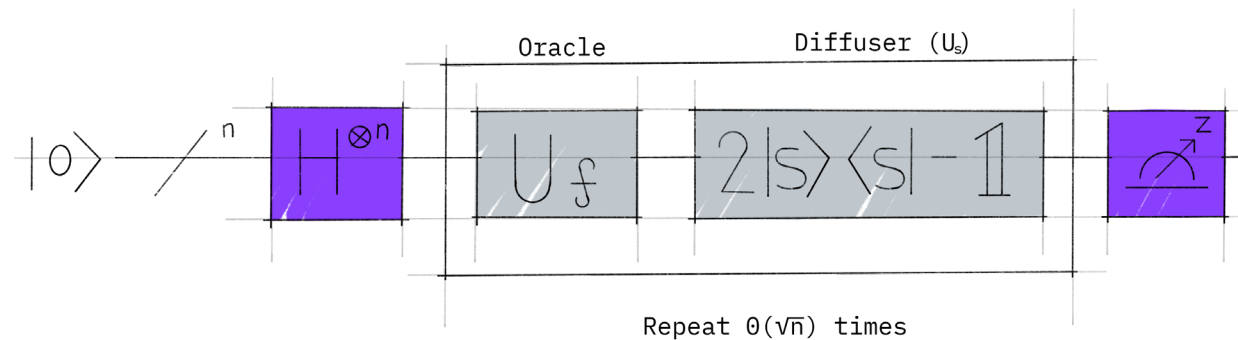
Grover's Algorithm

$$\frac{\pi}{2} = \theta + 2\theta \cdot k$$

$$k = \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N}$$

For Large N

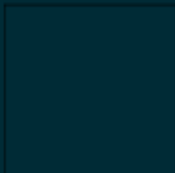
$$\theta \approx \arcsin \frac{1}{\sqrt{N}} \approx \frac{1}{\sqrt{N}}$$



Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm Example – Light Out!

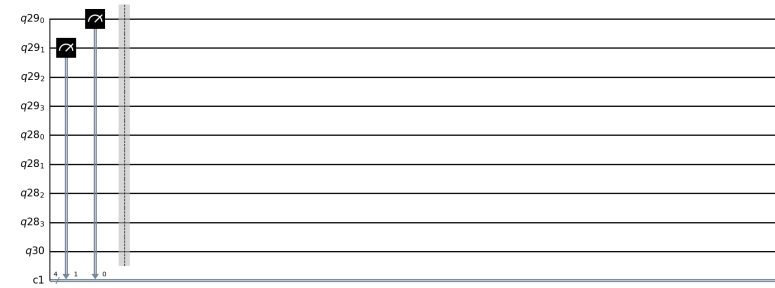
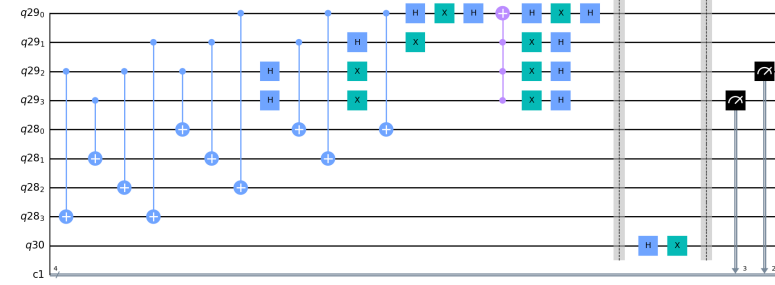
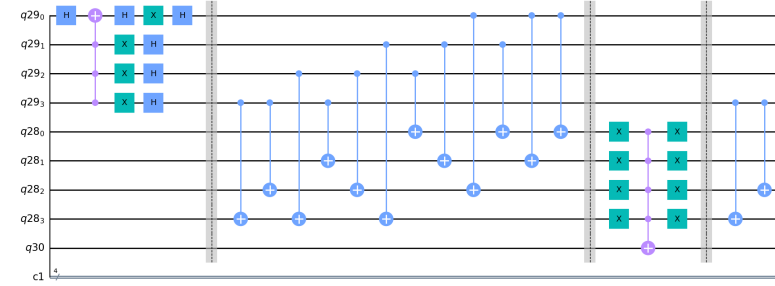
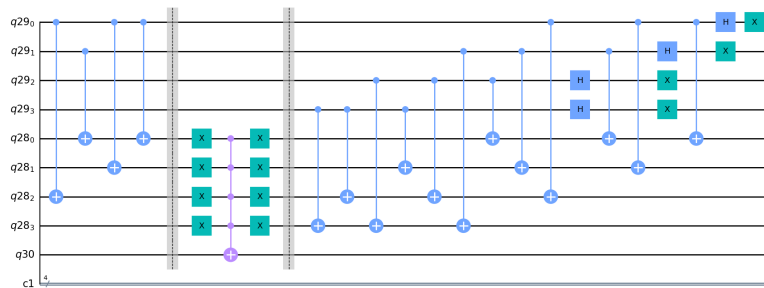
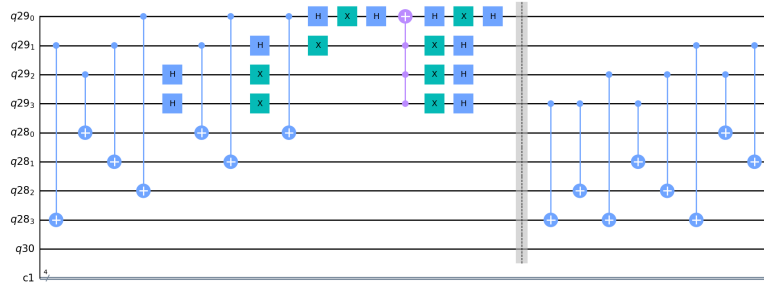
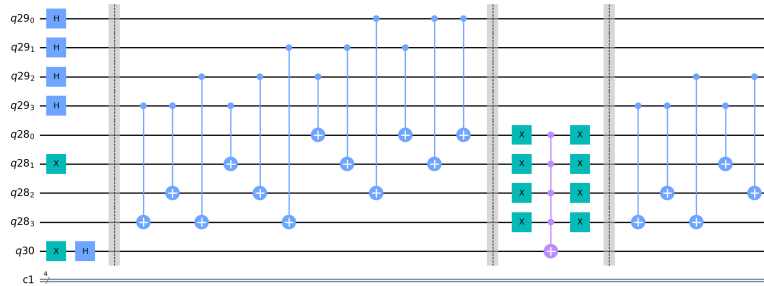
1		2		3		4	
							
							



Quantum Computing and Qiskit Practice

Quantum Algorithms

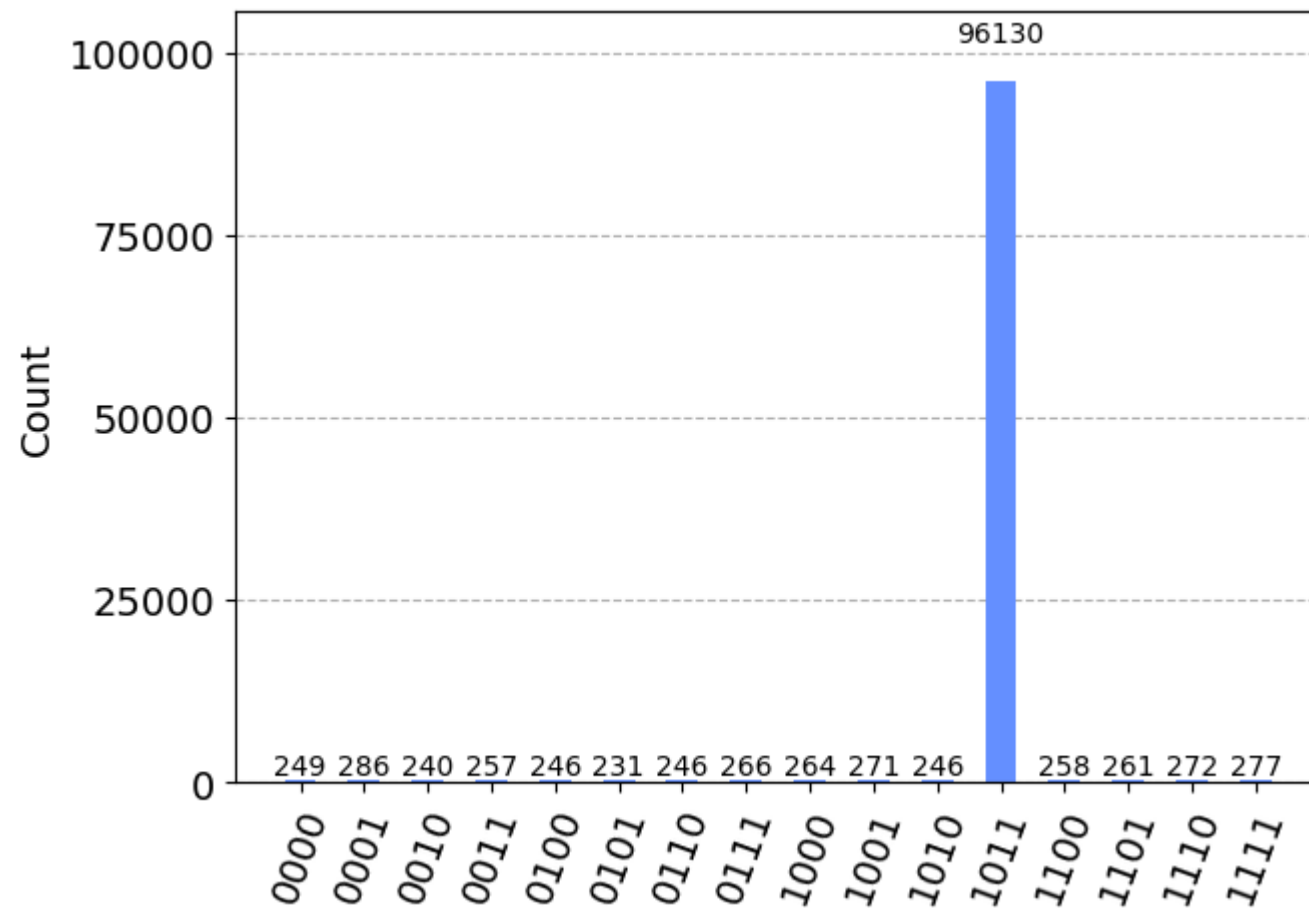
Grover's Algorithm Example – Light Out!



Quantum Computing and Qiskit Practice

Quantum Algorithms

Grover's Algorithm Example – Light Out!



Quantum Computing and Qiskit Practice

Quantum Algorithms

Bernstein-Vazirani Algorithm

$f: \{0, 1\}^n \rightarrow \{0, 1\}$ where $f(x) = x \cdot a \pmod 2$ with $a \in \{0, 1\}^n$

$$\begin{aligned} &|0\rangle^{\otimes n} \xrightarrow{\text{Hadamard Gate}} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \xrightarrow{\text{Oracle}(|x\rangle \rightarrow (-1)^{f(x)}|x\rangle)} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \\ &\xrightarrow{\text{Hadamard Gate}} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} \sum_{y=0}^{2^n-1} (-1)^{f(x)+x \cdot y} |y\rangle \xrightarrow{f(x)=x \cdot a \pmod 2} \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} \sum_{y=0}^{2^n-1} (-1)^{x \cdot (a \oplus y)} |y\rangle \end{aligned}$$

$$a \oplus y = 1 \rightarrow 0$$



Quantum Computing and Qiskit Practice

Quantum Algorithms

Bernstein-Vazirani Algorithm

